Threaded BST Predecessor and Successor

1. Problem Statement

Write a C++ program to find and display the inorder predecessor and successor of nodes in a Threaded Binary Search Tree (TBST). Handle boundary cases where the minimum node has no predecessor and the maximum node has no successor. Display the predecessor and successor for every node in the tree and verify the results against the inorder traversal sequence.

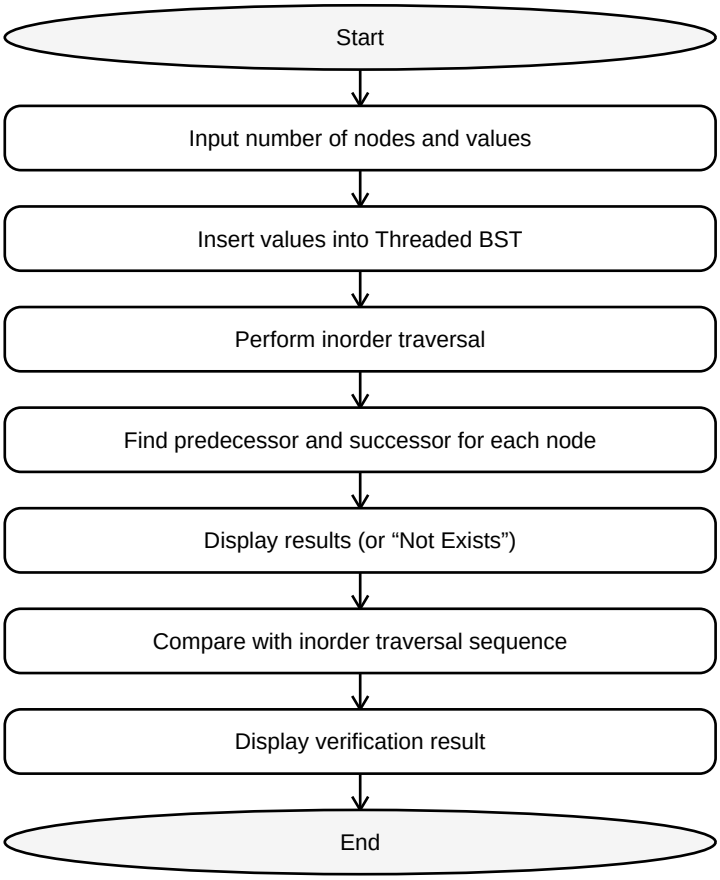
2. Objectives

- To understand the concept of Threaded Binary Search Tree.
- To implement a Threaded BST using C++.
- To find the inorder predecessor and successor of every node.
- To handle minimum and maximum nodes correctly.
- To display the inorder traversal of the tree.
- To verify the results against the inorder traversal sequence.
- To improve programming and problem-solving skills.

3. Algorithm

1. Start the program.
2. Define a Node structure containing data, left and right pointers, and thread flags.
3. Accept the number of nodes and their values from the user.
4. Insert the values into the Threaded BST.
5. Perform inorder traversal and store the values.
6. Find the predecessor and successor of every node.
7. Display Not Exists when a predecessor or successor is unavailable.
8. Compare the results with the inorder traversal sequence.
9. Display the verification result.
10. Stop the program.

4. Flowchart



5. Conclusion

The Threaded BST Predecessor and Successor program demonstrates how to find the inorder predecessor and successor of every node in a Threaded Binary Search Tree using C++. It handles the minimum and maximum nodes correctly and verifies the results against the inorder traversal sequence. This project helps in understanding threading, tree traversal, pointers, and binary search tree operations.